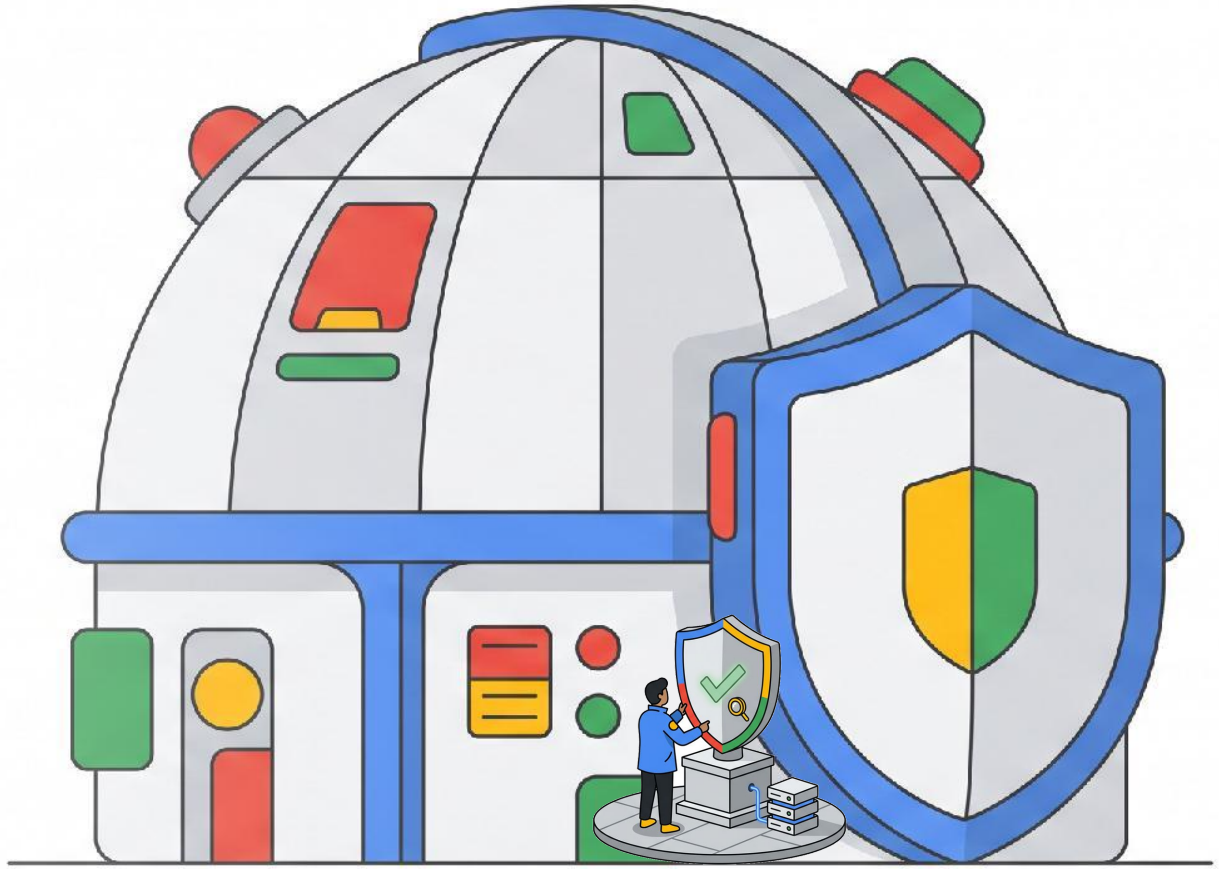
SECURITY BENCHMARK

CodeMender best practices

43 runs across Python, Node.js & Java (Benchmark)



EVALUATION METRICS

Four discoveries redefine CodeMender deployment

100%

Python GT recall

11/11 vulns, 0% FP (Benchmark)

118

Peak Node.js bugs

19 Crit, 58 High (Benchmark)

90

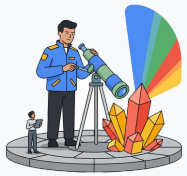
Peak Java bugs

8 Crit, 36 High (Benchmark)

9.0x

Deep mode uplift

10 to 90 Java bugs (Benchmark)



100% ground-truth recall

On Python ref-codemender-test, c3_hybrid_max hits 11/11 recall with 0% false positives.



4.9x–9.0x deep mode lift

Concurrent --deep workers jump findings from 24 to 118 (Node.js) and 10 to 90 (Java).



No literal filenames in -c

Avoid literal file.ext in --deep -c prompts so Tier 0 pruning never drops your codebase.

Golden evaluation metrics rule: Pair --deep mode with semantic c3_hybrid_max context and expanded config.yaml.

TIMEOUT BOTTLENECK

Avoid turn exhaustion & literal filename traps

ANTI-PATTERN: LITERAL FILE TRAP

Turn limits & literal -c trap



- Quick mode caps out at 16–24 JS and 10–13 Java findings
- Writing literal requirements.txt in --deep -c yields 0 bugs (py_13)
- Default config.yaml ignores *.xml, *.yml, and *.pug files

BEST PRACTICE: SEMANTIC --DEEP

Semantic context + 16 workers

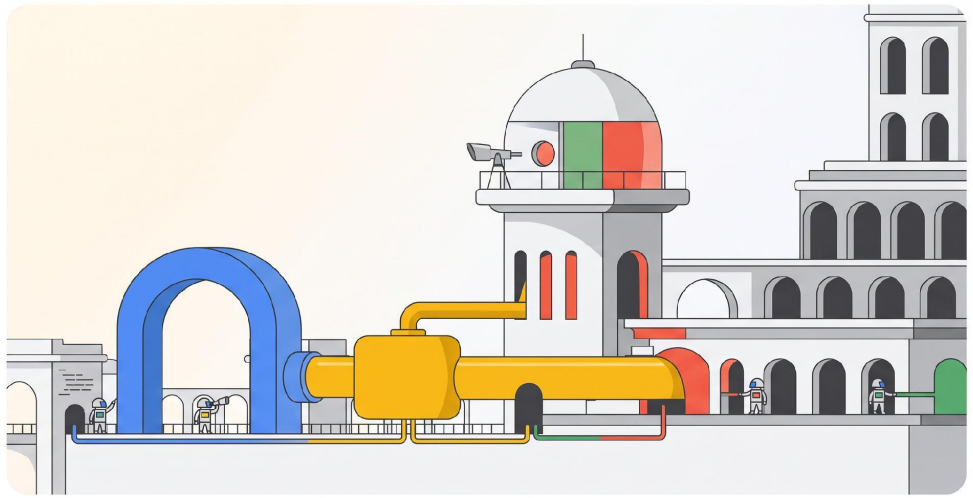


- Use semantic manifest descriptions in -c (0 literal file.ext)
- Reaches 100% (11/11) Python GT recall in 76.3s (py_12)
- Unlocks 118 Node.js bugs (4.9x) and 90 Java bugs (9.0x)

Why literal filenames break --deep: Regex scoping + Tier 0 lockfile pruning drops app.py (0% vs 100% recall).

TAINT PIPELINE

Enabling --deep unlocks multi-file taint tracking





Cross-file sink verification

Traces untrusted HTTP inputs across Python, TypeScript, and Java files to sinks.

Stage 1 — Route & entry enumeration (PROJECT_FILE_INDEX.md)

Maps Python WSGI rules, Express routes, and Spring @RequestMapping controllers.

Stage 2 — Cross-file taint pipeline (16 parallel workers)

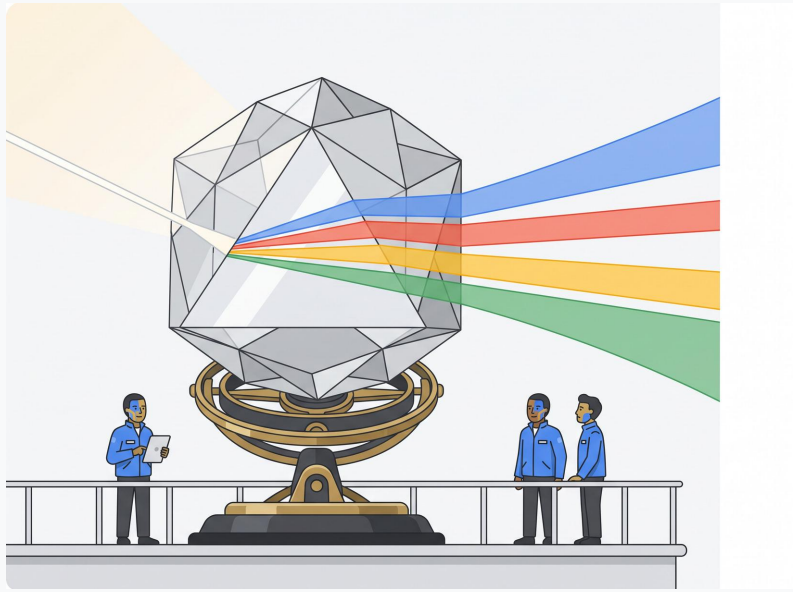
Follows tainted parameters through auth filters, services, and MyBatis XML mappers.

Stage 3 — Sink & safe wrapper check (0% false positives)

Verifies SQL/NoSQL parameterization and safe_join confinement without false alarms.

REASONING ENGINE

Tri-language model comparison across 43 runs





Frontier reasoning depth

Gemini 3.8 Flash leads Node.js (118 bugs); 4 Argon leads Java (90 bugs) and zero-shot Python (90.9%).

Model & Mode	Target ecosystem	Bugs	Crit / High	Recall / CWE	Sec
3.8 Flash (Quick c3)	Python WSGI (2f)	13	4 / 6	100% (11/11)	231
4 Argon (--deep c3)	Python WSGI (2f)	11	4 / 4	100% (11/11)	76
4 Argon (Quick c0)	Python WSGI (2f)	10	3 / 5	90.9% (10/11)	307
3.5 Flash (Quick c3)	Python WSGI (2f)	12	3 / 5	100% (11/11)	203
3.8 Flash (--deep c3)	Node.js bench (16f)	118	19 / 58	35 CWEs	224
3.8 Flash (--deep c3)	Node.js high (35f)	116	20 / 49	31 CWEs	536
4 Argon (--deep c3)	Java bench (20f)	90	8 / 36	29 CWEs	327
3.8 Flash (--deep c3)	Java bench (20f)	80	7 / 32	29 CWEs	253

Tri-language highlight: 100% (11/11) Python GT recall in 76s, 118 Node.js bugs, and 90 Java bugs.

CONTEXT BLUEPRINT

Hybrid -c context unlocks 100% Python GT recall



Zero context baseline

72.7% Py / 24 JS / 10 Java

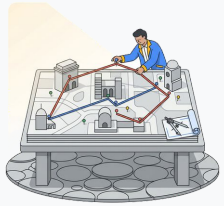
No -c flag; misses Werkzeug CVEs, Debugger PIN RCE, and Cookie flags.



SCA dependency manifest

100% Py / 25 JS / 13 Java

Supplies semantic manifest versions (Werkzeug 2.2.0, jsonwebtoken, MyBatis).



SAST framework topology

90.9% Py / 35 JS / 29 Java

Maps WSGI handlers, Express routes, Spring controllers, and XML mappers.



Hybrid max context brief

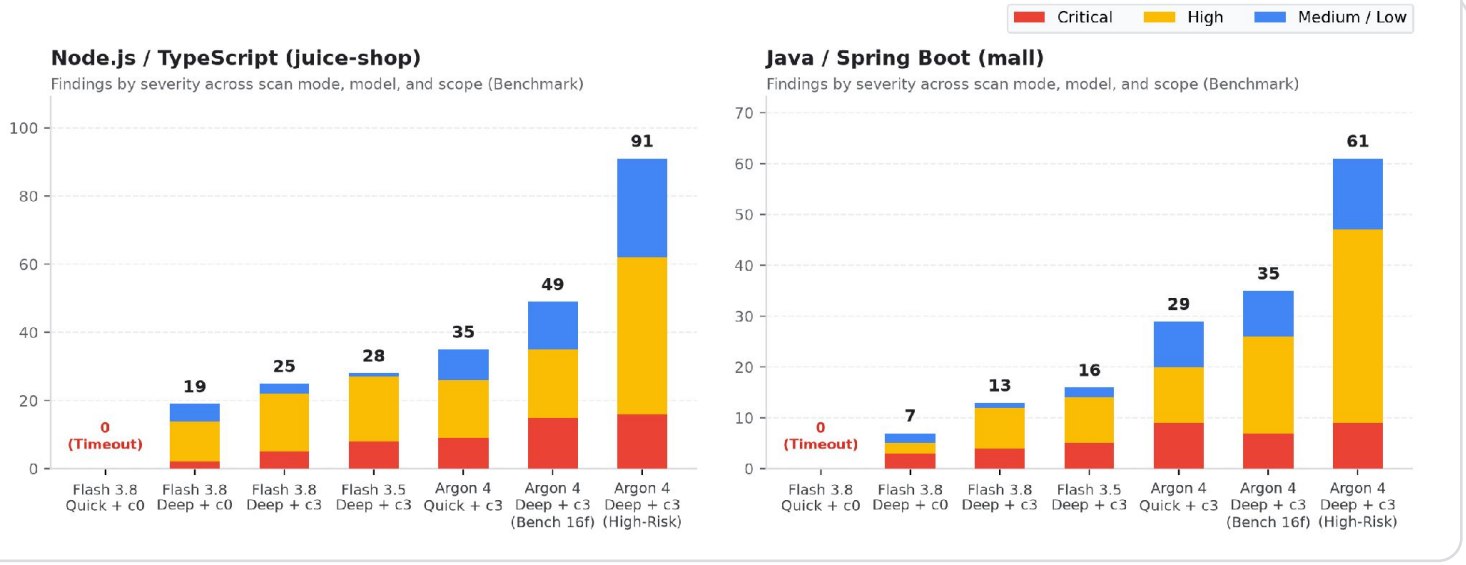
100% Py / 118 JS / 90 Java

Combines semantic SCA versions, SAST sinks, and safe_join control checks.

Context blueprint rule: Use semantic manifest descriptions (zero literal file.ext) to preserve 100% recall in --deep.

ABLATION GROWTH

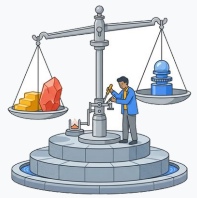
Model, mode & context compound





Compounding security gains

Ablation growth across 43 runs: --deep gives 4.9x–9.0x lift while c3_hybrid_max hits 11/11 Python GT recall.



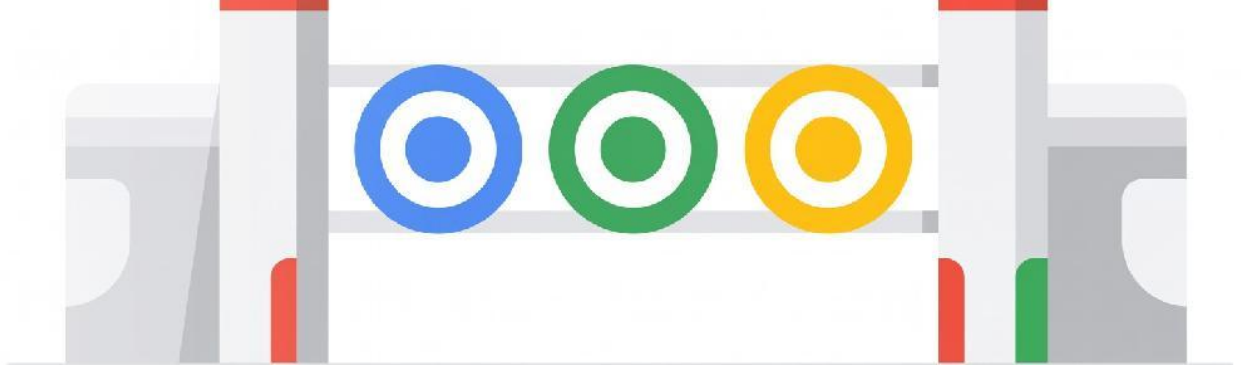
Zero false-positive control

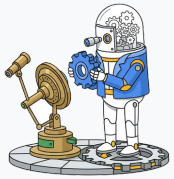
Across all 13 Python runs, 0 runs falsely flagged the safe_join (/files/secure-download) negative control.

43-run takeaway: Combining --deep + c3_hybrid_max hits 100% Python GT recall, 118 JS & 90 Java bugs.

DIRECTORY SCOPING

Match scan mode to repository scale





Microservice scope (1–5 files)

Python WSGI · 1–3 min

Use Quick + c3_hybrid_max or Argon --deep for 100% (11/11) GT recall & full SCA.



Curated service (15–50 files)

Node.js / Java · 3–6 min

Optimal directory scoping with PROJECT_FILE_INDEX.md (118 JS / 90 Java bugs).



Full core monolith

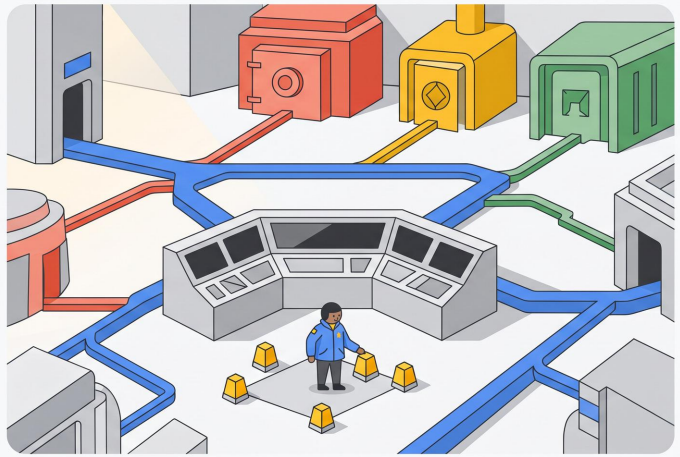
120–300+ files · 10–15 min

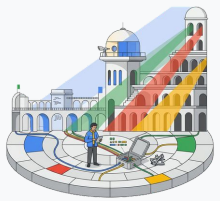
Exclude generated ORM POJOs (mall-mbg) and frontend specs to avoid dilution.

Scope profile	Files scanned	Recommended mode & model	Peak findings / Recall	Wall time	Primary CI/CD use case
Python WSGI micro	2 files	Quick or Argon --deep + c3	13 bugs (100% 11/11 GT)	76s–231s	Microservice & PR gate
Node.js bench scope	16 files	3.8 Flash (--deep w=16 t=8)	118 bugs (19 Crit / 58 High)	223.7s	Service-level security gate
Java bench scope	20 files	4 Argon (--deep w=16 t=8)	90 bugs (8 Crit / 36 High)	326.8s	Multi-module release gate
High-risk scale-out	35–48 files	3.8 Flash (--deep w=16 t=8)	116 JS / 59 Java bugs	536s–672s	Nightly full-surface audit

TYPESCRIPT ROUTES

Express best practices capture 118 findings





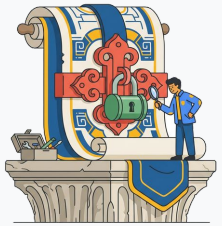
Express route tainting

Target typescript routes/.ts, lib/insecurity.ts, and views/.pug while excluding UI bundles.

Vulnerability class (CWE)	Bugs	Representative files & sinks discovered (Benchmark)
CWE-94 / 1336 Code & SSTI RCE	14	views/promotionVideo.pug (Pug SSTI), b2bOrder.ts (vm)
CWE-89 SQL Injection	8	routes/search.ts, login.ts (Sequelize raw interpolation)
CWE-943 NoSQL Injection	9	routes/showProductReviews.ts, trackOrder.ts (MarsDB)
CWE-918 Server-Side Request Forgery	5	routes/profileImageUrlUpload.ts (unvalidated HTTP fetch)
CWE-22 / 611 Path Traversal & XXE	12	routes/fileUpload.ts (unzipper Zip Slip & libxmljs XXE)
CWE-347 / 327 JWT & Crypto Flaws	15	lib/insecurity.ts (RS256->HS256 confusion, MD5, keys)
CWE-639 / 862 Broken Access Control	21	routes/basket.ts (IDOR), dataErasure.ts, updateProductReviews
CWE-79 / 601 XSS & Open Redirect	16	routes/redirect.ts, userProfile.ts (reflected XSS)

MYBATIS MAPPERS

Spring Boot & MyBatis rules uncover 90 findings



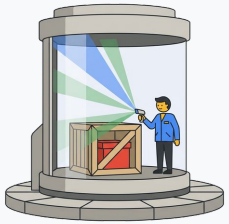
MyBatis XML SQL injection

Add .xml to config.yaml and scan resources/dao/*.xml to catch ${} SQLi in OmsOrderDao.xml (8 Critical).



Missing endpoint authorization

Pair portal controllers with SecurityConfig to expose unauthenticated admin routes and order IDOR (36 High).



Config secrets & OSS uploads

Add .yml to config.yaml to audit JWT secrets in application.yml, MinioController XSS, and Alipay callbacks.

Java golden scope rule: Exclude mall-mbg POJOs, add .xml/.yml to config.yaml, and generate PROJECT_FILE_INDEX.md.

PIPELINE AUTOMATION

Standardize PR gates and nightly deep audits

Production CLI profile (pipeline.py wrapper)

1. Configure CM_HOME/config.yaml & PROJECT_FILE_INDEX.md

include: [.py, .txt, .js, .ts, .java, .json, .xml, .yml, .pug, .hbs]

2. Build semantic hybrid context (-c with 0 literal file.ext)

```
export CM_CTX="Read file index; correlate manifest CVEs + SAST sinks."
```

3. Execute parallel deep scan (native tools, no --enable-bash)

```
cm -y --sandbox=false find . --deep --deep-workers 16 --deep-max-turns 8 -c "$CM_CTX"
```



Automated CI/CD security gate

Pipeline automation: keep --enable-bash off for 1.87x faster parallel worker completion.

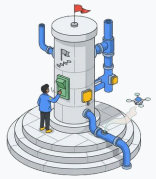


SQLite & JSON reporting

Export cm report -f json and state.db findings to block PR merges on Critical/High CWEs.

ROLLOUT ROADMAP

Five rules unlock peak recall across 3 stacks



Match mode to repo scale

Use Quick+c3 on 1–5 file microservices (100% Py GT) and --deep on 10–300+ file repos.



Zero literal filenames in -c

Use semantic manifest descriptions in --deep -c so Tier 0 pruning never drops code.



Extend config.yaml & index

Add .xml, .yml, .json, .pug, .txt to config.yaml and generate PROJECT_FILE_INDEX.md.

Rollout dimension	Anti-pattern to avoid	Recommended golden standard	Verified 43-run impact
Prompt syntax (-c)	Literal file.ext in --deep -c (py_13: 0%)	Semantic manifest description + versions	0% -> 100% (11/11) Python GT
Analysis mode	Quick mode on large multi-file repos	--deep --deep-workers 16 --deep-max-turns 8	4.92x JS (118) & 9.0x Java (90)
Config & extensions	Default config.yaml (misses XML/YAML)	Add .xml, .yml, .json, .pug + FILE_INDEX	Unlocks MyBatis SQLi & SSTI
Precision control	Unbounded noisy prompts	c3_hybrid_max + safe wrapper verification	100% recall with 0% FP (safe_join)